



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Satellite Data Visualization Platform for Weather and Climate Pattern Analysis

A CASE STUDY ANALYSIS REPORT

Submitted in the partial fulfilment for the Course of

CSA1012-Software Engineering

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence & Machine Learning

Submitted by

Y.Navya(192425177)

Under the Supervision of

Dr.K.Anita Davamani



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST-2026



Engineer to Excel

Table Of Contents

S.No	Table Of Contents	Page No.
1.	Problem Analysis	3-5
2.	Project Structure Design	5-8
3.	Git Repository Initialization	8-9
4.	Git Branching Strategy	10-11
5.	Version Control Workflow	11-13
6.	Commit History Analysis	14-15
7.	Docker Environment Design	15-116
8.	Dockerfile Development	16-17
9.	Docker Compose Design	17-19
10.	Container Deployment and Testing	19-21
11.	Benefits of Git and Docker	22-23
12.	Challenges and Improvements	25-27
13.	Conclusion	28

1. Problem Analysis

1.1 Problem Statement

The Satellite Data Visualization Platform for Weather and Climate Pattern Analysis is designed to make satellite-based weather and climate information easier to access, process, visualize, and analyze.

Meteorological organizations and space agencies generate large amounts of satellite data containing information about temperature, precipitation, cloud cover, vegetation, atmospheric conditions, and other environmental parameters. However, this information is often stored in complex scientific formats and requires specialized software and technical knowledge.

The proposed platform provides a user-friendly web interface through which users can view satellite data using interactive maps, charts, time-series visualizations, and multiple data layers. It reduces the amount of manual processing required by researchers and makes satellite-based information accessible to students, planners, government agencies, disaster management teams, and other non-expert users.

The system follows a visual data-processing approach in which raw data is acquired, processed, analyzed, and converted into meaningful information. This follows the general principle that raw visual data must be processed before it can provide useful analytical information.

1.2 Project Objectives

The major objectives are:

1. Develop a data ingestion pipeline to retrieve satellite imagery from publicly available sources such as NASA and NOAA.
2. Develop an interactive map-based visualization module.
3. Provide historical and real-time weather visualization.
4. Implement time-series analysis for temperature, rainfall, and cloud-cover data.

5. Detect important weather and climate anomalies.
6. Allow users to compare multiple satellite data layers.
7. Provide region-specific filtering.
8. Allow users to export processed data and visualizations.
9. Provide a simple web interface for non-technical users.
10. Create a scalable architecture for future satellite-data sources and predictive analytics.

1.3 Functional Requirements

The system shall:

- Retrieve satellite data from external data sources.
- Store and manage processed satellite datasets.
- Display weather and climate information on interactive maps.
- Display historical data using time-series charts.
- Allow users to select a geographical region.
- Allow users to select a date or date range.
- Display temperature, precipitation, cloud-cover, and vegetation information.
- Overlay multiple data layers.
- Identify unusual weather or climate patterns.
- Export processed datasets.
- Provide a web-based user interface.
- Provide APIs for communication between frontend and backend components.

1.4 Non-Functional Requirements

The system should provide:

- **Usability:** Simple interface for non-expert users.
- **Performance:** Fast visualization of selected datasets.
- **Scalability:** Ability to add more satellite sources.
- **Reliability:** Stable processing and visualization.
- **Portability:** Ability to run on different systems using Docker.

- **Maintainability:** Modular source-code structure.
- **Security:** Secure handling of API keys and configuration.
- **Availability:** Web-based access to the platform.

1.5 Expected Outcomes

The expected outcomes are:

1. Improved accessibility to satellite-based climate data.
2. Faster identification of weather patterns and anomalies.
3. Better disaster-management decision support.
4. Reduced manual processing for researchers.
5. Improved long-term climate trend analysis.
6. Increased use of satellite data in agriculture and urban planning.
7. Better academic and public engagement with climate information.
8. A scalable platform capable of supporting additional data sources.

2. Project Structure Design

A suitable repository structure for the project is:

satellite-data-visualization/

```
|
| └── frontend/
|   ├── public/
|   ├── src/
|   │   ├── components/
|   │   ├── pages/
|   │   ├── maps/
|   │   ├── charts/
|   │   └── services/
|   ├── package.json
|   └── Dockerfile
```

```
|
|
|— backend/
|   |— app/
|   |   |— api/
|   |   |— services/
|   |   |— models/
|   |   |— processing/
|   |   |— utils/
|   |— requirements.txt
|   |— Dockerfile
|
|
|— data/
|   |— raw/
|   |— processed/
|   |— sample/
|
|
|— notebooks/
|   |— climate_analysis.ipynb
|
|
|— tests/
|   |— test_api.py
|   |— test_processing.py
|   |— test_visualization.py
|
|
|— docs/
|   |— architecture.md
|   |— workflow.md
|
|
|— .gitignore
|— .env.example
|— docker-compose.yml
```

|— README.md
|— LICENSE

2.1 Folder Explanation

frontend/

Contains the web interface of the platform. It contains maps, charts, dashboards, filters, and other user-interface components.

backend/

Contains the server-side application. It handles satellite-data retrieval, processing, APIs, anomaly detection, filtering, and data export.

backend/app/api/

Contains API endpoints used by the frontend.

backend/app/services/

Contains services responsible for retrieving satellite data and communicating with external APIs.

backend/app/processing/

Contains data-cleaning, transformation, analysis, and anomaly-detection functions.

backend/app/models/

Contains data models used by the application.

data/raw/

Stores raw satellite datasets during development.

data/processed/

Stores processed datasets ready for visualization.

data/sample/

Contains small sample datasets used for testing.

notebooks/

Contains experimental data-analysis notebooks.

tests/

Contains automated tests for backend processing, APIs, and visualization-related functionality.

docs/

Contains project documentation, architecture diagrams, and workflow explanations.

Dockerfile

Defines how individual application components are converted into Docker images.

docker-compose.yml

Defines and runs multiple services together.

.gitignore

Specifies files and folders that Git should not track.

.env.example

Provides sample environment variables without exposing actual credentials.

README.md

Provides project description, installation instructions, execution steps, and usage information.

3. Git Repository Initialization

Git is used to maintain the source code and track every important change made to the project.

3.1 Repository Initialization Steps

Create the project directory:

```
mkdir satellite-data-visualization
```

```
cd satellite-data-visualization
```

Initialize Git:

```
git init
```

Create the initial project files and folders.

Check the repository status:

`git status`

Add files:

`git add .`

Create the first commit:

`git commit -m "Initial project structure"`

Create the main branch:

`git branch -M main`

If a remote repository is created:

`git remote add origin <repository-url>`

Push the project:

`git push -u origin main`

3.2 Essential Git Files

.git/

Contains Git's internal repository information, including commits, branches, references, and configuration.

.gitignore

Prevents unnecessary or sensitive files from being committed.

Example:

`.env`

`__pycache__/`

`node_modules/`

`*.pyc`

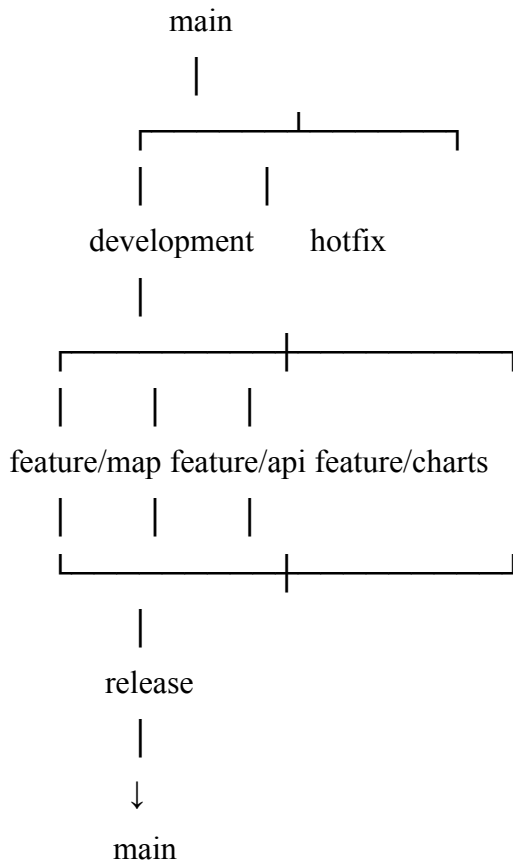
`data/raw/`

README.md

Describes the project and explains how to install and execute it.

4. Git Branching Strategy

A Git Flow-based branching strategy is suitable for this project.



4.1 Main Branch

The main branch contains stable and tested versions of the project.

4.2 Development Branch

The development branch contains integrated features before they are released.

4.3 Feature Branches

Individual features are developed using separate branches.

Examples:

feature/satellite-ingestion

feature/interactive-map

feature/time-series

feature/anomaly-detection

feature/data-export

4.4 Release Branch

A release branch is created when the system is ready for final testing.

Example:

release/v1.0

4.5 Hotfix Branch

Hotfix branches are created to correct critical problems in the production version.

Example:

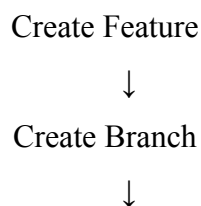
hotfix/map-loading-error

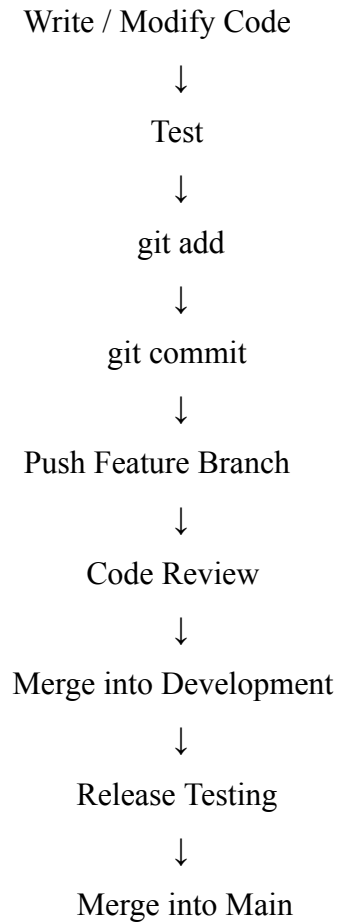
4.6 Justification

This branching model is appropriate because the platform contains multiple independent modules such as data ingestion, visualization, APIs, anomaly detection, and export functionality. Developers can work on separate features without disturbing the stable version.

5. Version Control Workflow

The project follows the workflow:





5.1 Create Feature Branch

```
git checkout development
git pull origin development
git checkout -b feature/interactive-map
```

5.2 Add Changes

After implementing the map module:

```
git status
git add frontend/src/maps/
```

5.3 Commit Changes

```
git commit -m "Add interactive satellite map visualization"
```

5.4 Push Branch

`git push -u origin feature/interactive-map`

5.5 Merge Feature

After testing:

`git checkout development`

`git pull origin development`

`git merge feature/interactive-map`

Push the updated development branch:

`git push origin development`

5.6 Conflict Resolution

If Git reports a conflict:

`git status`

Open the conflicting file and manually select the correct version.

Then:

`git add .`

`git commit -m "Resolve merge conflict in map module"`

5.7 Repository Synchronization

Before starting new work:

`git pull origin development`

After completing work:

`git push origin feature/interactive-map`

These operations keep the local and remote repositories synchronized.

6. Commit History Analysis

A meaningful commit history for the project can be:

a21f8d1 Initial project structure
b34c7e2 Add backend API framework
c45d9f3 Implement satellite data ingestion
d56e1a4 Add interactive map visualization
e67f2b5 Add climate time-series charts
f78g3c6 Implement anomaly detection
g89h4d7 Add region filtering
h90i5e8 Add data export functionality
i01j6f9 Add Docker configuration
j12k7g0 Add Docker Compose services
k23l8h1 Add automated tests
l34m9i2 Update project documentation

6.1 Good Commit Message Practices

Commit messages should:

- Be short and meaningful.
- Describe the actual change.
- Use an action-oriented format.
- Avoid vague messages such as changes, update, or done.
- Represent one logical change per commit.

Examples:

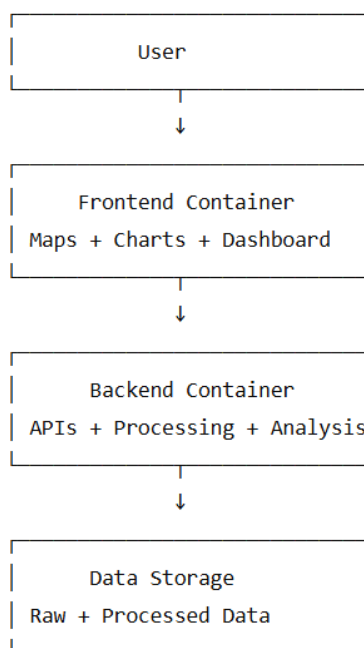
Add satellite data ingestion service
Add temperature time-series chart
Fix map layer rendering issue
Implement region filtering
Add Docker deployment configuration

A clear commit history makes it easier to understand project development, locate bugs, revert changes, and maintain the software.

7. Docker Environment Design

Docker is suitable for this project because the platform contains multiple components with different software dependencies.

The major components are:



7.1 Components Requiring Containerization

1. Frontend application
2. Backend API
3. Database/storage service, if used
4. Supporting processing services, if required

7.2 Advantages of Docker

Docker provides:

- Consistent development environments.

- Dependency isolation.
- Easy deployment.
- Portability.
- Simple application startup.
- Reproducible environments.
- Easier scaling.
- Separation between application components.

8. Dockerfile Development

A Dockerfile can be created for the backend.

8.1 Backend Dockerfile

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

8.2 Explanation of Instructions

FROM

FROM python:3.12-slim

Specifies the base Python image.

WORKDIR

WORKDIR /app

Sets /app as the working directory.

COPY

COPY requirements.txt .

Copies the dependency file into the container.

RUN

RUN pip install --no-cache-dir -r requirements.txt

Installs required Python libraries.

COPY

COPY app ./app

Copies the backend source code into the container.

EXPOSE

EXPOSE 8000

Documents that the backend listens on port 8000.

CMD

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

Starts the backend server when the container runs.

9. Docker Compose Design

Docker Compose is used to manage multiple containers as one application.

Example docker-compose.yml:

services:

 backend:

 build: ./backend

 container_name: satellite-backend

 ports:

 - "8000:8000"

environment:

- DATA_SOURCE=\${DATA_SOURCE}
- API_KEY=\${API_KEY}

volumes:

- ./data:/app/data

networks:

- satellite-network

frontend:

build: ./frontend

container_name: satellite-frontend

ports:

- "3000:3000"

depends_on:

- backend

networks:

- satellite-network

networks:

satellite-network:

volumes:

satellite-data:

9.1 Services

backend

Runs the Python API and data-processing services.

frontend

Runs the web interface containing maps, charts, filters, and dashboards.

9.2 Networking

Both containers are connected to:

satellite-network

This allows the frontend and backend to communicate through the Docker network.

9.3 Volumes

The data directory is mounted so that processed data can persist outside the container.

9.4 Environment Variables

API keys and configuration values are stored in environment variables rather than directly inside source code.

Example:

```
API_KEY=your_api_key  
DATA_SOURCE=NASA
```

Actual secret values should not be committed to Git.

9.5 Dependencies

The frontend depends on the backend:

```
depends_on:  
  - backend
```

This establishes the startup dependency between the services.

10. Container Deployment and Testing

10.1 Build the Containers

From the project root:

```
docker compose build
```

10.2 Start the Application

```
docker compose up
```

For background execution:

```
docker compose up -d
```

10.3 Check Running Containers

`docker compose ps`

Expected result:

NAME	STATUS
satellite-backend	running
satellite-frontend	running

10.4 Test Backend

Open:

`http://localhost:8000`

The backend should respond successfully.

10.5 Test Frontend

Open:

`http://localhost:3000`

The satellite visualization dashboard should appear.

10.6 Test Application Functions

The following functions should be tested:

Test	Expected Result
Open Dashboard	Dashboard loads
Select region	Selected region is displayed
Select date range	Corresponding data is displayed

Enable temperature layer	Temperature information appears
Enable precipitation layer	Precipitation information appears
Display time series	Climate trend chart appears
Compare Layers	Multiple layers are displayed
Export data	Processed data is downloaded
Restart Containers	Application starts successfully

10.7 Docker Logs

Backend logs can be checked using:

`docker compose logs backend`

Frontend logs:

`docker compose logs frontend`

10.8 Stop the Application

`docker compose down`

For the technical report, screenshots should be captured showing:

1. `git init`
2. `git status`
3. Git branch creation
4. Git commits
5. Git merge
6. Git log
7. Docker image building
8. `docker compose up`

9. Running containers
10. Browser displaying the application

11. Benefits of Git and Docker

11.1 Benefits of Git

Collaboration

Multiple developers can work on different features using separate branches.

Version Management

Git maintains a complete history of changes.

Backup

Remote Git repositories provide a backup of the source code.

Branching

Feature development can be isolated from the stable application.

Error Recovery

Previous versions can be restored when problems occur.

Maintainability

Clear commits make it easier to understand the evolution of the project.

11.2 Benefits of Docker

Portability

The application can run consistently on different computers.

Dependency Management

All required libraries can be installed inside containers.

Deployment

The application can be started using a small number of Docker commands.

Isolation

Frontend and backend dependencies remain separated.

Scalability

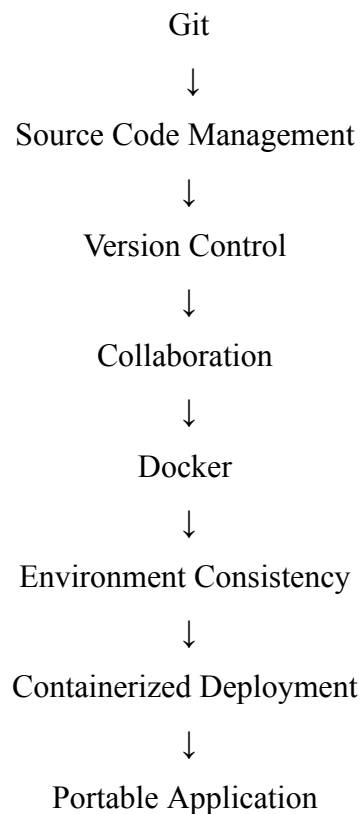
Additional containers can be created when application workload increases.

Reproducibility

The same Docker configuration can be used by developers, testers, and deployment environments.

11.3 Combined Benefits

Git and Docker together provide:



Therefore, Git manages how the project changes, while Docker manages how the project runs.

12. Challenges and Improvements

12.1 Challenges

1. Large Satellite Datasets

Satellite datasets can be very large and difficult to process.

Solution: Use preprocessing, data compression, caching, and selective downloading.

2. Different Data Formats

Satellite data may be available in different scientific formats.

Solution: Create a common processing layer that converts datasets into standardized formats.

3. API Availability

External satellite APIs may have request limits or temporary failures.

Solution: Implement caching, retry mechanisms, and error handling.

4. Processing Time

Large datasets may require significant processing time.

Solution: Use asynchronous processing and optimized data pipelines.

5. Docker Image Size

Scientific Python libraries can increase container size.

Solution: Use slim base images and install only required dependencies.

6. Git Merge Conflicts

Multiple developers may modify the same files.

Solution: Use feature branches, small commits, regular synchronization, and code reviews.

7. Sensitive API Credentials

Satellite services may require API keys.

Solution: Store credentials in environment variables and exclude .env files using .gitignore.

8. Visualization Performance

Large geographic datasets may slow down browser visualization.

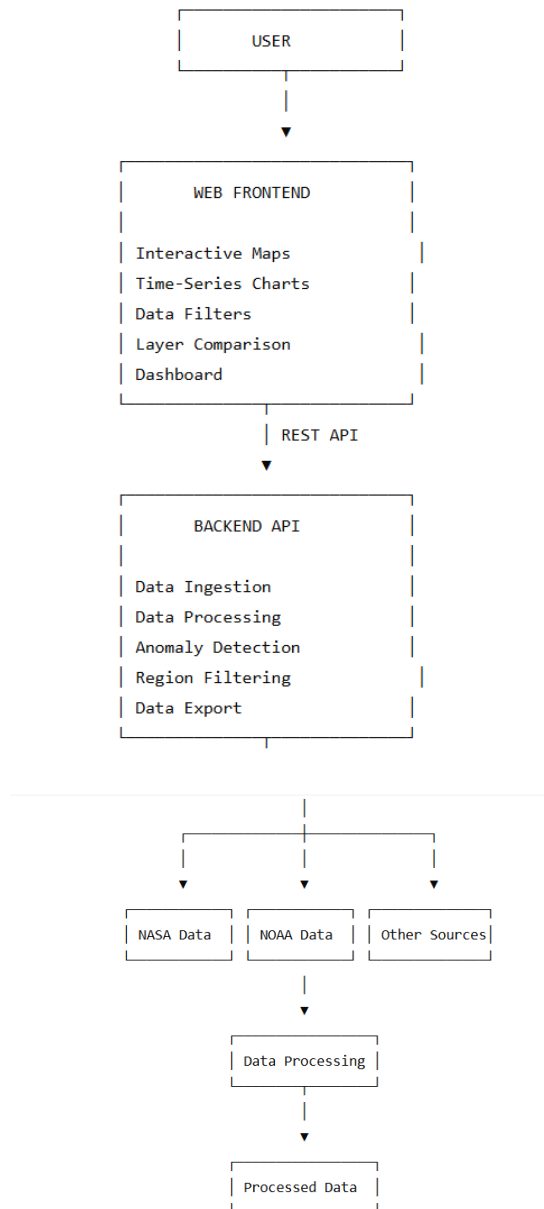
Solution: Use data aggregation, tiled maps, caching, and optimized visualization formats.

12.2 Future Improvements

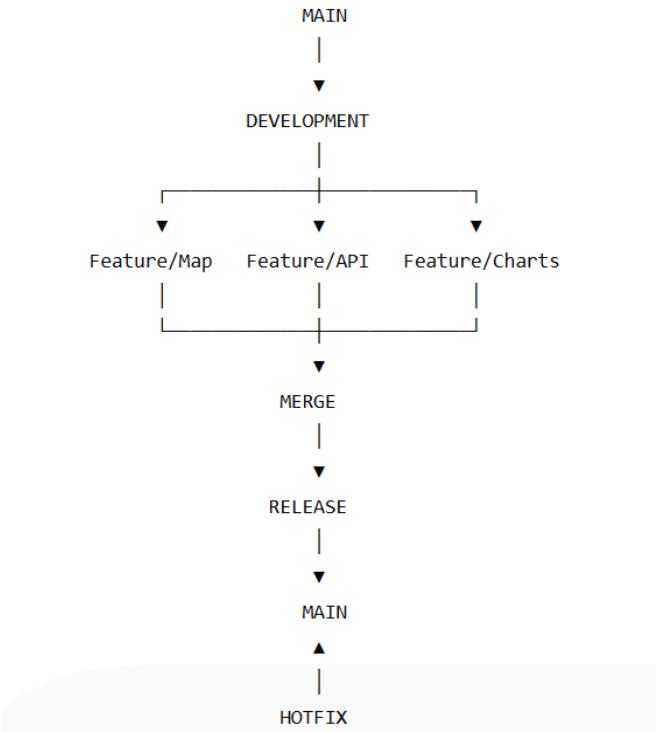
Future versions can include:

1. Machine-learning-based weather prediction.
2. Automated cyclone detection.
3. Drought prediction.
4. Flood-risk prediction.
5. AI-based anomaly detection.
6. More NASA and NOAA datasets.
7. Mobile application support.
8. User authentication.
9. Real-time weather alerts.
10. Cloud deployment.
11. Kubernetes-based scaling.
12. Advanced climate forecasting.
13. Automated CI/CD pipelines.
14. Automated Docker image building.
15. Monitoring and logging systems.

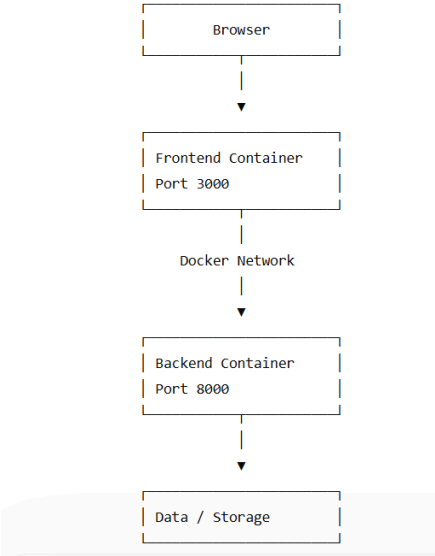
Project Architecture Diagram



Git Workflow Diagram



Docker Deployment Architecture



Conclusion

The Satellite Data Visualization Platform provides an accessible solution for transforming complex satellite weather and climate datasets into meaningful visual information. The system combines satellite-data ingestion, processing, interactive maps, time-series analysis, anomaly detection, region filtering, and data export into a single platform.

Git provides structured version control, collaboration, branching, commit history, and recovery mechanisms. Docker provides a consistent and portable environment for running the frontend, backend, and supporting services. Docker Compose further simplifies the management of these components as a single application.

The combination of Git and Docker makes the proposed platform easier to develop, test, maintain, collaborate on, and deploy. The architecture also provides a foundation for future improvements such as AI-based weather prediction, automated anomaly detection, cloud deployment, and real-time climate monitoring.